

NP-Completeness and Reduction Techniques

From 3-SAT to Graph Problems

Most. Sonia Khatun

Green University of Bangladesh

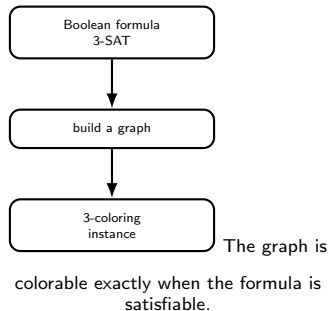
Goal of the Demo

Core question. How do we prove that a problem is computationally hard?

- ▶ We start from a known hard problem.
- ▶ We transform its instances into another problem.
- ▶ The transformation must preserve YES/NO answers.

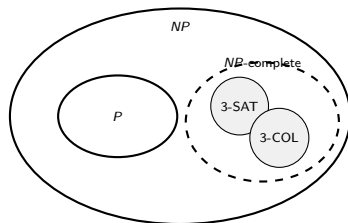
Main simulation.

$$3\text{-SAT} \leq_p 3\text{-COLORABILITY}$$



P, NP, and NP-Complete

- ▶ **P**: problems solvable in polynomial time.
- ▶ **NP**: YES answers have certificates verifiable in polynomial time.
- ▶ **NP-hard**: at least as hard as every problem in NP.
- ▶ **NP-complete**: both in NP and NP-hard.



If one NP-complete problem is in P, then $P = NP$

Example certificates:

- ▶ 3-SAT: a truth assignment.
- ▶ 3-COLOR: a color for each vertex.
- ▶ CLIQUE: a set of selected vertices.

Polynomial-Time Reduction

A reduction

$$A \leq_p B$$

means there is an efficient transformation f such that

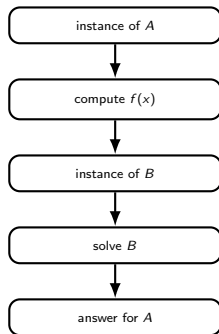
$$x \in A \iff f(x) \in B.$$

Interpretation. A solver for B would also solve A after the transformation.

Direction matters

To prove target problem B is hard:

known NP-complete $A \leq_p$ target B .



The Two Problems

3-SAT

Given a Boolean formula in CNF with exactly 3 literals per clause:

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m.$$

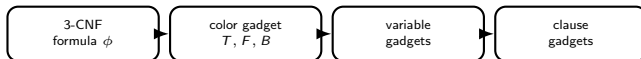
Question: is there a truth assignment satisfying all clauses?

3-COLORABILITY

Given a graph G , can we color every vertex using only three colors so that adjacent vertices always have different colors?

Theorem: $3\text{-SAT} \leq_p 3\text{-COLORABILITY}$.
--

Reduction Plan: Build a Graph from a Formula



- ▶ The color gadget gives fixed meanings to the three colors.
- ▶ Each variable gadget forces x_i and $\neg x_i$ to take opposite truth values.
- ▶ Each clause gadget is colorable only when at least one of its literals is true.

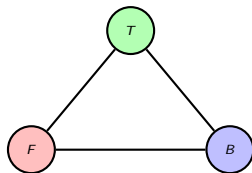
ϕ is satisfiable $\iff$ constructed graph G_ϕ is 3-colorable.

Simulation Step 1: Color Gadget

Create three special vertices T , F , B and connect them as a triangle.

- ▶ T means the true color.
- ▶ F means the false color.
- ▶ B means base/helper color.

Because they form a triangle, all three must receive different colors.



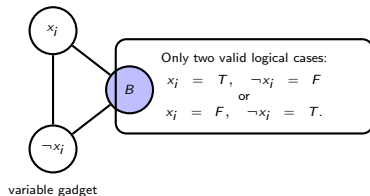
palette triangle

Simulation Step 2: Variable Gadget

For each variable x_i , create two vertices x_i and $\neg x_i$.

- ▶ Connect x_i to $\neg x_i$.
- ▶ Connect both to the base vertex B .

Then x_i and $\neg x_i$ cannot use the base color and cannot have the same truth color.



Simulation Step 3: Clause Gadget Interface

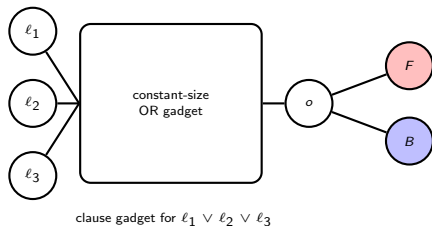
For a clause

$$(\ell_1 \vee \ell_2 \vee \ell_3),$$

attach a constant-size OR gadget to the three literal vertices.

Required property

The gadget can be 3-colored exactly when at least one input literal has color T .



Clause Gadget Truth Table

ℓ_1	ℓ_2	ℓ_3	clause value	gadget colorable?
F	F	F	F	No
T	F	F	T	Yes
F	T	F	T	Yes
F	F	T	T	Yes
T	T	F	T	Yes
T	F	T	T	Yes
F	T	T	T	Yes
T	T	T	T	Yes

The only forbidden case is when all three literals are false.

Simulation Formula

Use the following small formula:

$$\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3).$$

Clauses

$$C_1 = (x_1 \vee \neg x_2 \vee x_3)$$

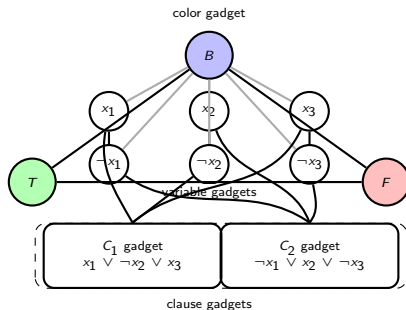
$$C_2 = (\neg x_1 \vee x_2 \vee \neg x_3)$$

One satisfying assignment

$$x_1 = T, \quad x_2 = T, \quad x_3 = F.$$

Then C_1 is true by x_1 , and C_2 is true by x_2 and $\neg x_3$.

Simulation Step 4: Assemble the Graph



Each clause gadget is connected only to the literal vertices appearing in that clause.

Simulation Step 5: Assignment $\Rightarrow$ Coloring

Take

$$x_1 = T, \quad x_2 = T, \quad x_3 = F.$$

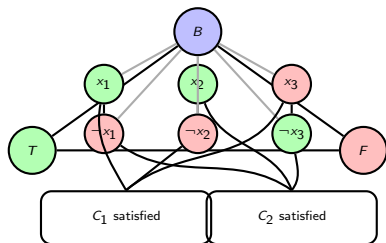
Color the literal nodes:

$$x_1 : T, \quad \neg x_1 : F$$

$$x_2 : T, \quad \neg x_2 : F$$

$$x_3 : F, \quad \neg x_3 : T$$

Every clause has at least one true literal.



Correctness of the Reduction

ϕ satisfiable $\Rightarrow G_\phi$ 3-colorable

- ▶ Color T, F, B in the palette triangle.
- ▶ For each variable, color the true literal with color T and the false literal with color F .
- ▶ Since each clause has a true literal, every clause gadget can be extended to a legal coloring.

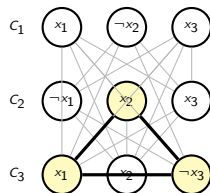
G_ϕ 3-colorable $\Rightarrow \phi$ satisfiable

- ▶ Each variable gadget forces exactly one of $x_i, \neg x_i$ to have color T .
- ▶ Set that literal to true.
- ▶ Each clause gadget being colorable implies at least one input literal is true.

ϕ is satisfiable $\iff G_\phi$ is 3-colorable.

Quick Second Example: $3\text{-SAT} \leq_p \text{CLIQUE}$

- ▶ Create one vertex for each literal occurrence.
- ▶ Put vertices in clause groups.
- ▶ Connect vertices from different clauses if they are not contradictory.
- ▶ Ask for a clique of size m .



One compatible literal from each clause forms a clique.

Takeaway and References

Takeaway

- ▶ NP-completeness is proved using polynomial-time reductions.
- ▶ The reduction must preserve YES/NO answers.
- ▶ Gadget reductions encode logical constraints inside graph structure.

References

- ▶ Michael Sipser, *Introduction to the Theory of Computation*.
- ▶ Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*.
- ▶ CMU 15-451/651 lecture notes on NP-completeness.
- ▶ Stanford CS103 slides on 3-SAT to 3-COLOR gadgets.

Thank you.